

REGULAR GRAMMAR

A Context-free grammar $G = (V, \Sigma, S, P)$ is called regular if every production is of the form :

$$A \rightarrow \alpha B$$

$$A \rightarrow \alpha$$

where $A, B \in V$ and
 $\alpha \in \Sigma^+$.

Grammar in which every production has one of the above form are equivalent to finite automata, and
for every Language L , $L \neq \{\epsilon\}$ can be generated by such a grammar

Sometimes the term regular is applied to grammars that do not restrict the form of reduction so severely.

A language is regular if and only if it can be generated, except possibly for the null string, by a grammar in which all productions look like

$$A \rightarrow xB$$

$$A \rightarrow x$$

where $A, B \in V$ and
 x is a non-null string of terminals.

Grammars of this type are also called **Linear**.

Regular Language to CFG

 - EQUIVALENCE OF CFG AND REGULAR LANG

1. One way : $R.E \rightarrow CFG$ (using Closure Properties of CFL)
2. 2nd way : $FA \rightarrow \text{Regular Grammar}$ (Simplest form of CFG, used only for Regular Languages)

First we discuss first way to see, Construction of CFG using closure properties is tedious. Consider the R.E :

$$bba(ab)^* + (ab + b\bar{a}b)^*ba$$

A literal application of the construction of CFG is tedious :

expression $ab + b\bar{a}b$ alone involve all three operations :

$$L_1 \cup L_2, L_1 \cdot L_2 \text{ \& } L_1^*$$

We don't really need to introduce separate variables for $\{a\}$ and $\{b\}$,
(Like in Thompson Construction method used to construct NFA from RE)

there are shortcuts to reduce the number of variables in the grammar as

in $bba(ab)^*$ can be converted directly to $S_1 \rightarrow S_1 ab \mid bba$

So, for $RE = bba(ab)^* + (ab + b\bar{a}b)^*ba$

CFG is

$$\begin{aligned} S &\rightarrow S_1 \mid S_2 \\ S_1 &\rightarrow S_1 ab \mid bba \\ S_2 &\rightarrow TS_2 \mid ba \\ T &\rightarrow bUb \mid ab \\ U &\rightarrow aU \mid \Lambda \end{aligned}$$

This is the required CFG of the given Regular Language expressed by R.E.

But this is not reversible. There is no algorithm that can be used to convert this CFG into RE or FA. That's why we prefer to write Regular Grammar for regular Language.

$$\begin{aligned} S_1 &\rightarrow S_1 ab \mid bba \\ S_1 &\Rightarrow S_1 ab \\ &\Rightarrow S_1 abab \\ &\Rightarrow S_1 ababab \\ &\Rightarrow S_1 (ab)^* \\ &\Rightarrow bba(ab)^* \end{aligned}$$

$$\begin{aligned} S_2 &\Rightarrow TS_2 \\ S_2 &\Rightarrow TTS_2 \\ &\Rightarrow T^* S_2 \\ &\Rightarrow T^* ba \\ T &\Rightarrow bUb \mid ab \\ T &\Rightarrow b\bar{a}b + ab \\ \Rightarrow S_2 &\Rightarrow T^* ba \\ &\Rightarrow (b\bar{a}b + ab)^* ba \end{aligned}$$

2. Second Alternative: $FA \rightarrow CFG(RG)$

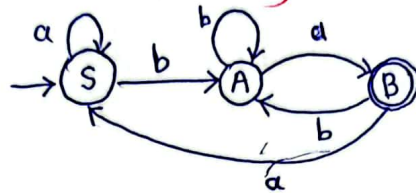
{ Starting from FA and then Convert FA to CFG (Regular Grammar and
Starting from Regular Grammar and then Convert it into FA (most probably NFA)

The resulting grammar is distinctive in two ways:

- The productions all have a very simple form, closely related to moves of the FA
- Construction is reversible, so that a CFG of this simple type (RG) can be used to generate a corresponding FA.

Example:

Consider the following FA:



Equivalent CFG (Regular Grammar):

$$G = (V, \Sigma, S, P)$$

$$V = Q = \{S, A, B\}$$

$$\Sigma =$$

$$S = S \text{ (start state)}$$

$$P: (FA)$$

$$\begin{array}{l} \delta(S, a) = S \Rightarrow S \rightarrow aS \\ \delta(S, b) = A \Rightarrow S \rightarrow bA \end{array} \quad \left| \quad \begin{array}{l} \delta(A, a) = B \Rightarrow A \rightarrow aB \\ \delta(A, b) = A \Rightarrow A \rightarrow bA \end{array} \right.$$

$$\begin{array}{l} \delta(B, a) = S \Rightarrow B \rightarrow aS \\ \delta(B, b) = A \Rightarrow B \rightarrow bA \end{array}$$

$$So \quad P = \begin{cases} S \rightarrow aS \mid bA \\ A \rightarrow aB \mid bA \\ B \rightarrow aS \mid bA \\ B \rightarrow \Lambda \end{cases}$$

As B is the final state of FA
 $F = \{B\}$

So, add $B \rightarrow \Lambda$ in the final CFG

This is reversible:

Suppose we have:

$$\begin{array}{l} S \rightarrow aS \mid bA \\ A \rightarrow aB \mid bA \\ B \rightarrow aS \mid bA \mid \Lambda \end{array}$$

Now Convert it into FA.
Rec

We have $S \rightarrow aS \mid bA$
 $A \rightarrow aB \mid bA$
 $B \rightarrow aS \mid bA \mid \Lambda$
 Convert into FA (reversal):

$$G = (V, \Sigma, S, P)$$

$$V = S, A, B$$

$$\Sigma = S \text{ (start var.)}$$

$$P: \begin{array}{l} S \rightarrow aS \quad A \rightarrow aB, B \rightarrow aS \\ S \rightarrow bA \quad A \rightarrow bA, B \rightarrow bA \\ B \rightarrow \Lambda \end{array}$$

Equivalent FA:

$$M = (Q, \Sigma, q_0, \delta, F)$$

$$Q = V = \{S, A, B\}$$

$$\Sigma =$$

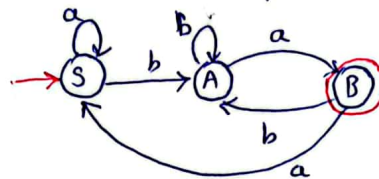
$$q_0 = \text{start variable } S$$

$$F = \{B\} \quad B \rightarrow \Lambda \text{ in Grammar shows } B \text{ is final state}$$

δ : For every production $T \rightarrow \alpha U$, there is a transition $\delta(T, \alpha) = U$

$$\begin{array}{l} \text{So, } \delta(S, a) = S \quad \delta(A, a) = B \quad \delta(B, a) = S \\ \delta(S, b) = A \quad \delta(A, b) = A \quad \delta(B, b) = A \end{array}$$

$B \rightarrow \Lambda$
only indicates
B is final
state.



Note: We cannot expect that M (result F&M) is an FA, because for some combinations of T and α , there may be either more than or fewer than ~~than~~ one

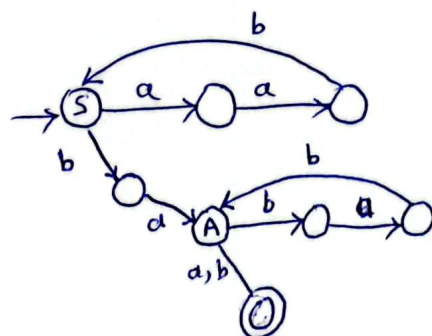
for which $T \rightarrow \alpha U$

RG $\rightarrow$ NFA (may or may not)

Example: Suppose we have

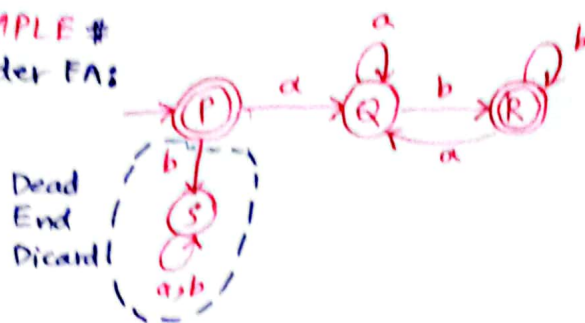
$$\begin{array}{l} S \rightarrow aabS \mid baA \\ A \rightarrow babA \mid a \mid b \end{array} \quad (\text{Also regular Grammar format})$$

Equivalent (now) NFA:



(This is not an FA)
its NFA.

EXAMPLE #
Consider FA:



Equivalent
Grammar
 $\Rightarrow$

$$G = (V, \Sigma, S, P)$$

$$V = (P, Q, R)$$

$$\Sigma =$$

$$S = P$$

P:

$$P \rightarrow aQ \mid \Lambda$$

$$Q \rightarrow aQ \mid bR$$

$$R \rightarrow aQ \mid bR \mid \Lambda$$

Right Linear and Left Linear Grammar

$$\begin{array}{l} \underline{\underline{RL}} \quad \left. \begin{array}{l} S \rightarrow aabS \mid baA \\ A \rightarrow babA \mid a \mid b \end{array} \right\} \begin{array}{l} S \rightarrow Saa \mid A bb \\ A \rightarrow Aa \mid a \end{array} \quad \underline{\underline{LL}} \end{array}$$

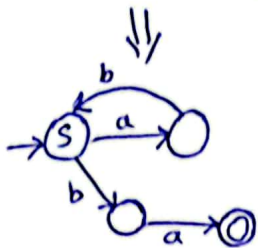
Example:

Construct NFA

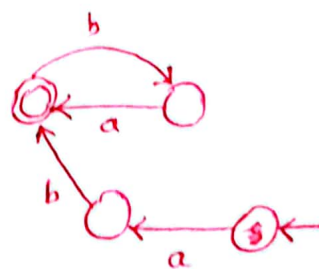
$$S \rightarrow Sba \mid ab$$

Take the right Linear form of above:

$$S \rightarrow abS \mid ba$$



1. Reverse the arrows
2. final to start
start to final



$$S \rightarrow ab \mid abS$$